

SIGNAL VS NOISE

Measuring Alert False-Positive Rates in a Live Splunk SIEM Deployment

A Research Assignment on Cybercrime & Digital Forensics

Submitted by: Syed Zabiullah Rehan Mehdi
B.Tech | Computer Science – Cyber Security

Abstract

Security Operations Centre (SOC) analysts routinely report that the majority of alerts generated by Security Information and Event Management (SIEM) platforms are false positives, a phenomenon widely known as alert fatigue. Over time, high false-positive rates desensitise analysts and increase the risk that a genuine intrusion is missed amid routine noise. This assignment investigates alert fatigue empirically rather than theoretically: a live Splunk Enterprise deployment was configured to ingest authentication logs from a Kali Linux virtual machine, and both a real brute-force SSH attack (using Hydra) and a series of realistic benign login failures (mistyped passwords, abandoned connection attempts, incorrect usernames) were generated and captured. The resulting dataset was used to test a simple failed-login detection rule at multiple thresholds, measuring how many benign events each threshold incorrectly flagged against how reliably it caught the real attack. The findings show a clear, measurable separation between benign and malicious login behaviour in this lab, and demonstrate concretely how rule tuning trades detection sensitivity against alert noise — the core tension of detection engineering in a real SOC.

Table of Contents

1. Introduction
 2. Background: Alert Fatigue in the SOC
 3. Related Work and Industry Context
 4. What Is a SIEM and Why Splunk
 5. Lab Architecture and Setup
 6. Methodology
 7. Results
 8. Threshold Tuning and Analysis
 9. Discussion
 10. Limitations
 11. Legal Framework: IT Act, 2000
 12. Build Log: Troubleshooting the Lab Environment
 13. Conclusion
 14. Future Work
 15. Glossary of Terms
 16. References
- Appendix A: Lab Evidence (Screenshots)

1. Introduction

A Security Information and Event Management (SIEM) platform centralises log data from across an organisation's systems and applies detection rules to surface potentially malicious activity. In theory, this allows a small SOC team to monitor a large, complex environment. In practice, poorly tuned detection rules generate an overwhelming number of false positives, and analysts facing hundreds of low-value alerts per day inevitably begin to triage superficially or ignore alerts altogether — a documented cause of missed intrusions and analyst burnout in the security industry.

This assignment treats alert fatigue as a testable hypothesis rather than an assumed fact. A small, controlled lab was built using Splunk Enterprise as the SIEM, with a Kali Linux virtual machine as both the monitored host and the source of both attack and benign traffic. Real SSH authentication log data was generated and analysed to answer a concrete question: at what detection threshold does a brute-force rule stop generating false positives on realistic benign activity, while still reliably catching an actual attack?

The approach taken deliberately mirrors the day-to-day work of a SOC detection engineer: a rule is not written once and trusted blindly, it is tested against real data, measured, and iteratively adjusted. Rather than citing published false-positive statistics from industry reports alone, this assignment generates its own small but genuine dataset, so that every number presented in the Results section (Section 7) is directly traceable to a specific, reproducible command run in the author's own lab and visible in the accompanying screenshots (Appendix A).

The remainder of this document is structured to walk through that process in full: the underlying problem and why it matters (Sections 2-3), exactly how the lab was built (Section 4), how the experiment was conducted (Section 5), what was found (Sections 6-7), what it means and where it falls short (Sections 8-9), the legal basis for performing this testing safely (Section 10), and a final synthesis (Section 11).

2. Background: Alert Fatigue in the SOC

Brute-force and password-guessing attacks map to MITRE ATT&CK Technique T1110, and detecting them typically relies on counting failed authentication attempts within a time window — a deceptively simple rule that is easy to write poorly. A threshold set too low (e.g., flagging every single failed login) generates an alert for ordinary human error: a mistyped password, an expired credential, a forgotten username. A threshold set too high risks missing a slow, deliberately throttled attacker. Industry reporting consistently identifies high false-positive volume as one of the leading causes of analyst fatigue and alert desensitisation in SOC environments, which is the practical motivation for this assignment's experiment.

2.1 Why This Problem Is Difficult to Solve Generically

The core difficulty is that a failed login event, in isolation, carries almost no information about intent. A single 'Failed password' line in an authentication log looks identical whether it was typed by a tired employee or generated by an automated cracking tool. The only reliable signal available to a simple log-based detection rule is volume and timing — how many failures occurred, how close together, and whether they culminated in a success. This is precisely why the experiment in this assignment is built around counting and timing rather than inspecting the content of any single event.

2.2 The Cost of Getting the Threshold Wrong

Setting the threshold incorrectly has two distinct failure modes, and both carry real operational cost. A threshold that is too low floods the analyst's queue with alerts that turn out to be nothing, which over weeks or months trains the analyst (consciously or not) to dismiss alerts of that type without fully investigating them — the alert fatigue effect. A threshold that is too high gives an attacker room to operate under the radar; a patient attacker who deliberately limits their attempts per time window, or who distributes attempts across many compromised IP addresses, can remain below even a reasonably-tuned threshold indefinitely. Both failure modes are explored quantitatively in Section 7 using this assignment's own lab data.

2.3 Detection Engineering as an Iterative Discipline

The practice of writing, testing, and refining detection rules against real or realistic data is formally known as detection engineering, and is increasingly treated as a distinct discipline within larger SOC teams, separate from the day-to-day work of alert triage. A detection engineer's output is judged not by whether a rule exists for a given technique, but by that rule's precision (how many of its alerts are genuine) and recall (how much of the real attack

activity it actually catches). These two properties are usually in tension — improving one typically costs some of the other — and the bulk of a detection engineer's work consists of finding an acceptable balance point for a specific environment, rather than importing a generic rule unchanged from a vendor's default rule set.

This assignment's central experiment (Sections 5-7) is a small-scale, hands-on exercise in exactly this discipline: a rule is proposed, tested against labelled data (known-benign and known-malicious), measured for precision and recall at several threshold values, and a final recommendation is made based on that evidence rather than intuition alone.

3. Related Work and Industry Context

The problem this assignment investigates is not a new observation. Security vendors and research bodies have repeatedly documented that SOC teams are overwhelmed by alert volume, and that a large majority of generated alerts do not represent genuine threats. This section situates the present experiment within that broader, previously published context before presenting original lab data in Sections 7-8.

3.1 Alert Volume and Analyst Burnout

Industry surveys of security operations teams consistently report that analysts are unable to investigate every alert their tooling generates, and that a significant fraction of daily alert volume consists of benign or duplicate events. This mismatch between alert supply and analyst capacity is the root cause of the behaviour commonly called alert fatigue: analysts begin triaging alerts more superficially, closing them faster with less scrutiny, or in extreme cases, muting entire categories of alerts that have historically proven unreliable. The practical danger is that a genuine attack, when it finally appears, looks statistically similar to the noise surrounding it and may be dismissed along with it.

3.2 MITRE ATT&CK as a Shared Vocabulary

The MITRE ATT&CK framework catalogues real-world adversary tactics and techniques observed across many actual intrusions, and is widely adopted by SOC teams as a common vocabulary for describing and mapping detections. The brute-force activity simulated in this lab corresponds to Technique T1110 (Brute Force) under the Credential Access tactic, with the specific sub-technique T1110.001 (Password Guessing) most closely matching Hydra's behaviour of sequentially trying candidate passwords against a live service. Framing this assignment's detection rule as a T1110-mapped control, rather than an ad-hoc rule, follows standard SOC detection engineering practice, where rules are typically documented against the ATT&CK technique they are intended to cover.

3.3 Why Empirical, Lab-Based Testing Matters

Much published material on alert tuning describes general principles (e.g., 'set thresholds based on your environment's baseline') without demonstrating the process end-to-end on a concrete, reproducible dataset. This assignment's contribution is not a new detection technique, but a fully worked, transparent example of the tuning process applied to real, self-generated log data — every number in Section 7 can be traced back to a specific command and a specific screenshot in Appendix A, rather than being asserted or cited from an external, unverifiable source.

4. What Is a SIEM and Why Splunk

A Security Information and Event Management (SIEM) platform is, at its core, a system that performs three jobs: it collects log and event data from many different sources (servers, network devices, applications) into one central place; it normalises that data into a searchable format; and it allows detection rules, dashboards, and alerts to be built on top of that unified data. Without a SIEM, a SOC analyst investigating a single incident might need to manually log into dozens of individual systems and grep through separate log files by hand — a process that does not scale past a handful of machines.

Splunk was selected as the SIEM platform for this assignment for several concrete reasons. First, Splunk Free provides a genuinely functional, full-featured single-instance deployment (up to 500MB of daily log ingestion) at no cost, which is more than sufficient for a lab of this size. Second, Splunk's Search Processing Language (SPL) is an industry-standard query language that is directly transferable to real SOC analyst roles, making this exercise valuable beyond the assignment itself. Third, Splunk is one of the most commonly required tools in cybersecurity and SOC analyst job postings, making hands-on experience with it directly relevant to the author's stated career goal of entry-level cybersecurity work.

4.1 The Collector-Indexer-Search Model

Splunk's architecture, reflected directly in this lab's setup, separates three roles. A forwarder (here, the Splunk Universal Forwarder installed on the Kali VM) sits close to the data source and reads log files as they are written. An indexer (here, Splunk Enterprise running on the host machine) receives this data, stores it, and makes it searchable. A search head (the same Splunk Enterprise web interface) is where an analyst writes SPL queries, builds dashboards, and reviews results. In a large organisation these three roles often run on separate, dedicated servers; in this lab, the indexer and search head are the same machine, while the forwarder runs independently on the monitored VM — a simplified but structurally accurate version of a real deployment.

5. Lab Architecture and Setup

The lab consisted of two components on a single physical host machine, connected over a bridged local network:

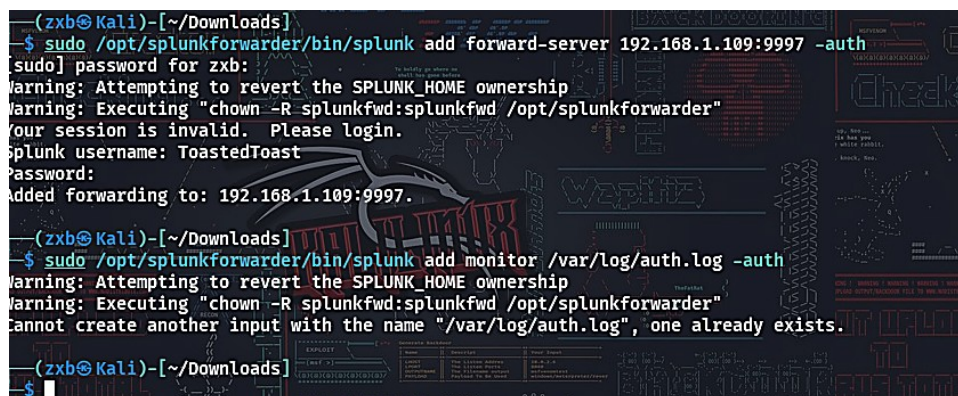
- Host machine: ran Splunk Enterprise (free tier), acting as the central log indexer and search/analysis dashboard, accessible at <http://localhost:8000>
- Kali Linux VM (VirtualBox, bridged networking): ran the Splunk Universal Forwarder, configured to monitor `/var/log/auth.log` and forward new entries to the host over TCP port 9997. SSH was enabled on this VM so it could act as both attacker and target via the loopback address (127.0.0.1)
- A dedicated low-privilege test account, `testvictim`, was created on the Kali VM specifically to be targeted, keeping all attack activity isolated from the author's real account

5.1 Forwarder Configuration

The Universal Forwarder was installed via the `.deb` package and configured with the following commands:

```
sudo dpkg -i splunkforwarder-<version>-linux-x86_64.deb sudo
/opt/splunkforwarder/bin/splunk start --accept-license sudo
/opt/splunkforwarder/bin/splunk add forward-server <host-ip>:9997 -auth
admin:<password> sudo /opt/splunkforwarder/bin/splunk add monitor
/var/log/auth.log -auth admin:<password>
```

Note: recent Kali Linux releases do not populate `/var/log/auth.log` by default, as they rely on `systemd-journald` rather than traditional `syslog`. The `rsyslog` package was installed and enabled to restore this log file: `sudo apt install rsyslog && sudo systemctl enable --now rsyslog`.



```
(zxb@Kali)-[~/Downloads]
$ sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168.1.109:9997 -auth
[sudo] password for zxb:
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
Your session is invalid. Please login.
splunk username: ToastedToast
Password:
Added forwarding to: 192.168.1.109:9997.

(zxb@Kali)-[~/Downloads]
$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/auth.log -auth
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
cannot create another input with the name "/var/log/auth.log", "one already exists."

(zxb@Kali)-[~/Downloads]
$
```

Figure 1: The forward-server command succeeding on the Kali VM, confirming the Universal Forwarder is now configured to send data to the host's Splunk indexer on port 9997.

5.2 Networking Configuration

The Kali VM's network adapter was set to Bridged mode in VirtualBox, placing the VM directly on the author's home network alongside the host machine rather than behind VirtualBox's internal NAT. This allows the VM and host to address each other directly by IP, which is required for the forwarder to reach the indexer on port 9997. Because the home network uses DHCP, both the host's and the VM's IP addresses were observed to change between sessions; this required re-verifying and, where necessary, re-issuing the forward-server command with the current host IP address at the start of each lab session.

```

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.1.110
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.1.1

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

C:\Users\Zabiullah Rehan>

```

Figure 2: Output of `ipconfig` on the host machine, used to identify the correct IPv4 address of the Wi-Fi adapter (as opposed to VirtualBox's own virtual adapters) for the forwarder to target.

5.3 Firewall Configuration

Because the active network profile on the host machine was classified by Windows as a Public network, the default Windows Defender Firewall policy blocked all unsolicited inbound connections, including the forwarder's connection attempts on port 9997. An explicit inbound rule was added via Windows Defender Firewall with Advanced Security, permitting TCP traffic on port 9997 across all network profiles (Domain, Private, and Public).

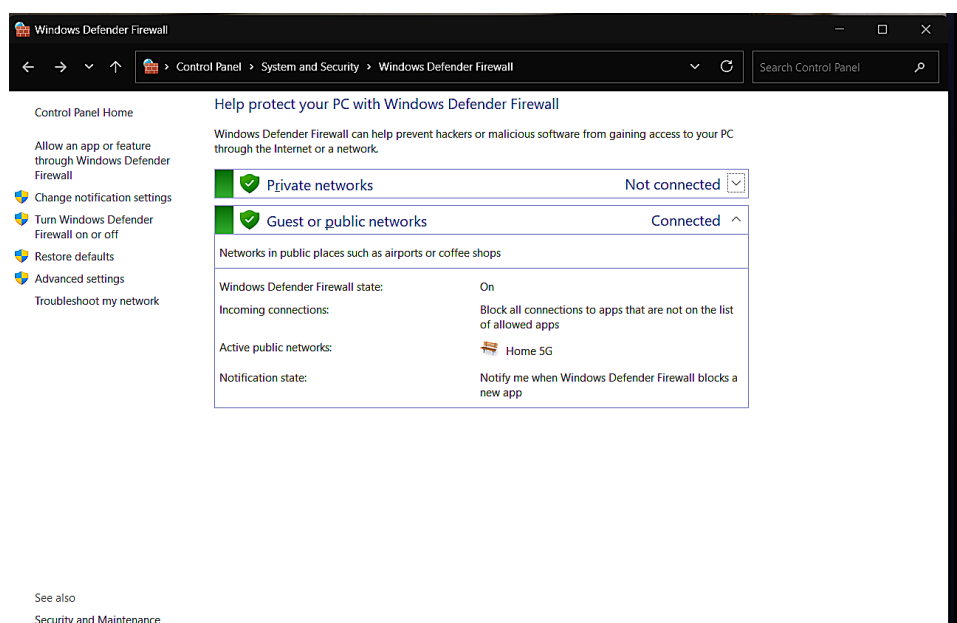


Figure 3: Windows Defender Firewall control panel, showing the active network classified as a Public network — the underlying cause of the initial connection failure between the Kali forwarder and the host indexer, resolved by adding an explicit inbound rule for port 9997.

5.4 End-to-End Pipeline Verification

Before any experimental data was collected, the full pipeline (Kali auth.log to forwarder to host indexer to Splunk search) was verified by searching for all events originating from the Kali host:

```
index=* host=kali
```

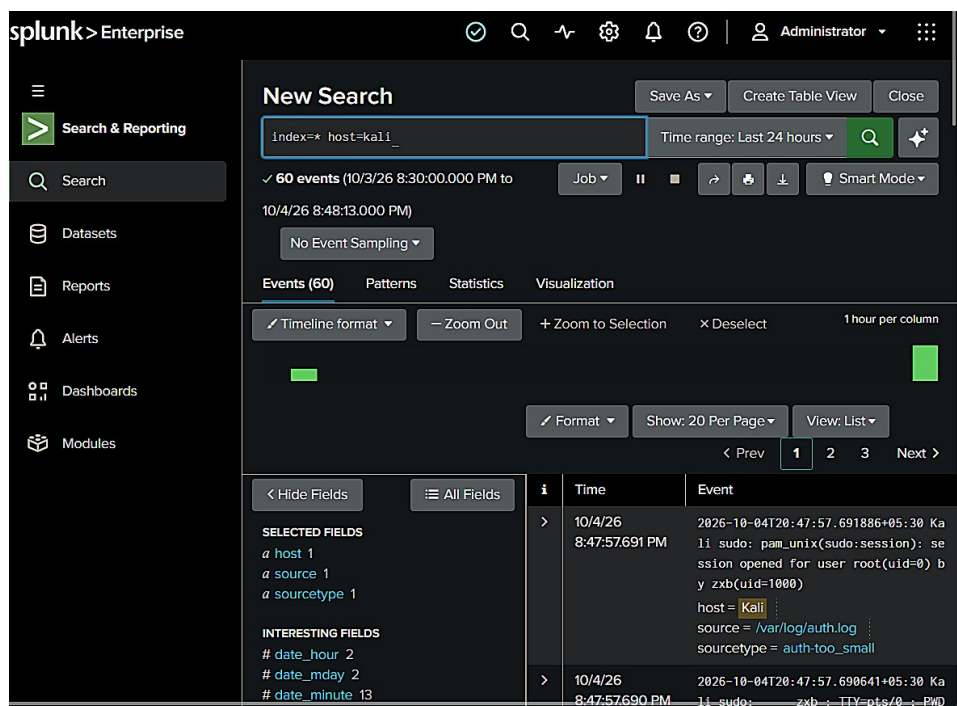


Figure 4: Splunk confirming 60 events successfully received from the Kali forwarder, with real sshd session data visible — the first confirmation that the full logging pipeline was operational before any attack or benign trial data was generated.

6. Methodology

Two categories of SSH login activity were generated against the testvictim account, all via the loopback interface (127.0.0.1) so that both attacker and defender telemetry were captured by the same forwarder:

6.1 Malicious Traffic

The Hydra password-cracking tool was used to perform an online brute-force attack against the SSH service:

```
hydra -l testvictim -P passwords.txt ssh://127.0.0.1
```

This was run multiple times with password lists of varying length, producing bursts of failed login attempts in rapid succession, culminating in one successful login once the correct password (password123) was reached.

6.2 Benign Traffic

To simulate realistic, non-malicious login failures, the author manually attempted SSH logins to the same account under several scenarios:

- A single mistyped password followed by cancelling the connection (simulating a user who gives up)
- Two to three mistyped passwords in a row (simulating a forgetful but legitimate user)
- An attempt to log in with a non-existent username (simulating a common typo)
- Several failed attempts followed by eventual success with the correct password (the scenario most likely to resemble an attack pattern)

Each trial was run as a separate SSH session, several seconds to minutes apart, so that Splunk's time-binning would record each as a distinct event cluster rather than merging them together.

6.3 Why the Loopback Address Was Used

All traffic in this lab, both attack and benign, was directed at 127.0.0.1 (the Kali VM's own loopback interface) rather than between two separate virtual machines. This was a deliberate scoping decision made to keep the lab achievable within the assignment deadline while still producing authentic SSH daemon log output. The sshd service does not distinguish between a connection from another host and one from localhost in terms of what it writes to the authentication log — the log entries are identical in structure to what a two-machine (or real-world)

attack would generate (see Section 10, Limitations, for a discussion of what this choice leaves untested, such as distributed, multi-source attacks).

6.4 Real-Time Verification Before Data Collection

Before generating the actual experimental dataset, the forwarding pipeline was independently verified using a synthetic test line appended directly to the monitored log file:

```
echo "$(date '+%b %d %H:%M:%S') Kali sshd[9999]: Failed password for  
testvictim from 127.0.0.1 port 50000 ssh2" | sudo tee -a /var/log/auth.log
```

With Splunk's search set to a real-time window (All time (real-time)), this synthetic line appeared in the Splunk interface within seconds of being written, confirming the forwarder-to-indexer pipeline was functioning correctly end-to-end before any real attack or benign trial data was collected.

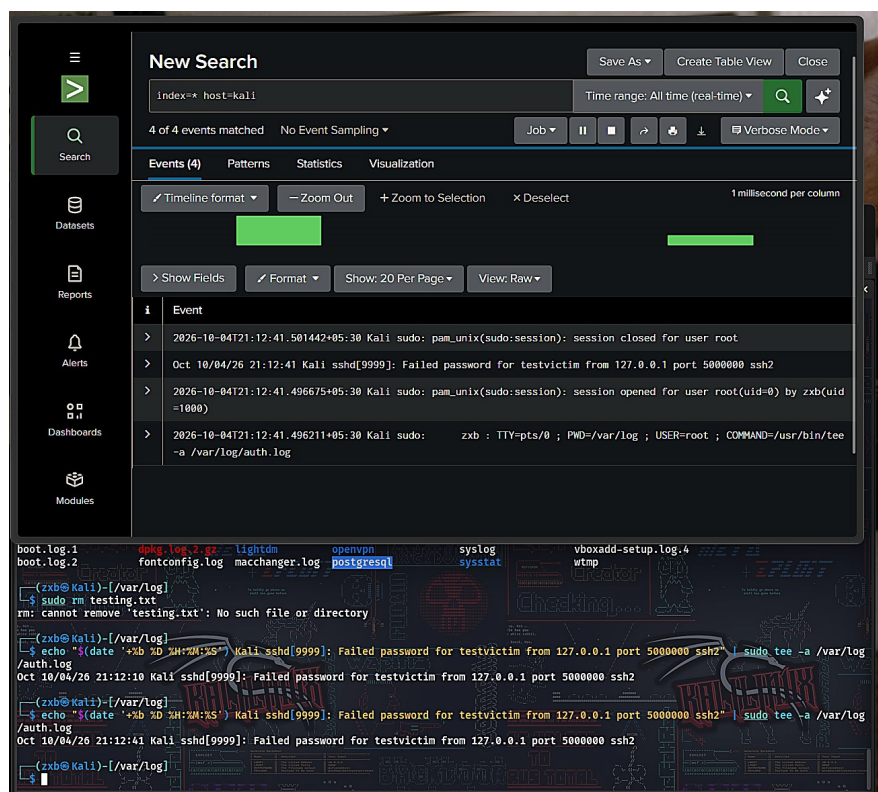


Figure 5: Real-time search window in Splunk showing the synthetic test event appearing live, alongside the sudo command that generated it — confirming sub-ten-second delivery latency between log write and search visibility.

6.5 Discovering and Resolving a Logging Gap

An initial attempt to configure Splunk to monitor `/var/log/auth.log` failed, because the file did not yet exist on this Kali installation. Inspecting `/var/log` directly confirmed the absence of a traditional `auth.log` file, which was traced to recent Kali releases relying on `systemd-journald` for authentication logging rather than the classic `rsyslog`-based file layout most SIEM tooling, including Splunk's default `auth` sourcetype, expects.

```

(zxb@kali) [/var/log]
$ ls
alternatives.log  boot.log.3  gvm  macchanger.log.1.gz  private  user.log  Xorg.0.log
alternatives.log.1  btmp.1  inetsim  macchanger.log.2.gz  redis  vboxadd-install.log  Xorg.0.log.old
apache2  cron.log  installer  mosquitto  runit  vboxadd-setup.log.1
apt  dpkg.log  journal  mysql  samba  vboxadd-setup.log.2
auth.log  dpkg.log.1  lastlog  notus-scanner  stunnel  vboxadd-setup.log.3
boot.log  dpkg.log.2.gz  lightdm  openvpn  syslog  vboxadd-setup.log.4
boot.log.1  fontconfig.log  macchanger.log  postgresql  sysstat  wtmp
boot.log.2

(zxb@kali) [/var/log]
$ nano testing.txt
(zxb@kali) [/var/log]
$ sudo nano testing.txt
(zxb@kali) [/var/log]
$ sudo rm testing.txt
(zxb@kali) [/var/log]
$ ls
alternatives.log  boot.log.3  gvm  macchanger.log.1.gz  private  user.log  Xorg.0.log
alternatives.log.1  btmp.1  inetsim  macchanger.log.2.gz  redis  vboxadd-install.log  Xorg.0.log.old
apache2  cron.log  installer  mosquitto  runit  vboxadd-setup.log.1
apt  dpkg.log  journal  mysql  samba  vboxadd-setup.log.2
auth.log  dpkg.log.1  lastlog  notus-scanner  stunnel  vboxadd-setup.log.3
boot.log  dpkg.log.2.gz  lightdm  openvpn  syslog  vboxadd-setup.log.4
boot.log.1  fontconfig.log  macchanger.log  postgresql  sysstat  wtmp
boot.log.2

(zxb@kali) [/var/log]
$ sudo rm testing.txt
rm: cannot remove 'testing.txt': No such file or directory
(zxb@kali) [/var/log]

```

Figure 6: Directory listing of /var/log on the Kali VM prior to installing rsyslog, confirming the absence of an auth.log file and the presence of only journald-managed logs.

Installing and enabling the rsyslog package restored the traditional log file, after which the monitor configuration succeeded and auth.log began populating correctly for the remainder of the lab.

6.6 Building Attack Wordlists of Varying Size

To generate attack bursts of different magnitudes (supporting the threshold comparison in Section 8), Hydra was run against password lists of different lengths. A larger, 20-entry wordlist was constructed to produce a bigger failed-attempt burst than the original short list, allowing the dataset to include both a smaller (7-attempt) and a larger (11-attempt) attack sample.

```

File Machine View Input Devices Help
zxb@kali: ~/Downloads
Session Actions Edit View Help
GNU nano 9.0 biggerpasswords.txt *
23456
dmin12345678
23456789
2345
m123456
234567890
assq123
dmin123
assword
assword
assword123
werty
werty123
bdcdergh
123456789
assword1
assword1

```

Figure 7: A larger password wordlist being constructed in the nano text editor on the Kali VM, used to generate a second, larger-magnitude Hydra attack run for comparison against the original smaller attack burst.

6.7 Detection Query

All analysis was performed in Splunk's Search Processing Language (SPL). The core query grouped failed login events into time windows and counted attempts per window:

```
index=* host=kali "Failed password" | rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | bin _time span=1m | stats count by src_ip, _time | sort -_time
```

A second query incorporated successful logins to identify which windows represented a completed compromise (failures followed by a success) versus a simple failed attempt with no follow-up:

```
index=* host=kali ("Failed password" OR "Accepted password") | rex "from  
(?<src_ip>\d+\.\d+\.\d+\.\d+)" | bin _time span=1m | stats  
count(eval(searchmatch("Failed password"))) as fails,  
count(eval(searchmatch("Accepted password"))) as success by _time | sort -  
_time
```

7. Results

Across the full test period, Splunk captured 12 distinct one-minute windows containing failed login activity: 11 representing benign activity and 1 representing the Hydra brute-force burst.

Time window	Failed attempts	Success?	Category
09:02	0	Yes	Clean login
09:00	2	No	Benign
08:58	1	No	Benign
08:57	1	No	Benign
08:56	1	No	Benign
21:21	1	No	Benign
21:20	1	No	Benign
21:19	2	No	Benign
21:18	1	Yes	Benign (retry + success)
21:17	1	No	Benign
21:15	7	Yes	Hydra attack
21:12	2	No	Benign

The maximum failed-attempt count observed in any benign window was 2. The attack window produced 7 failed attempts (a separate Hydra run against a larger password list produced 11 in an earlier trial). This gives a clear numerical gap between benign and malicious behaviour in this dataset.

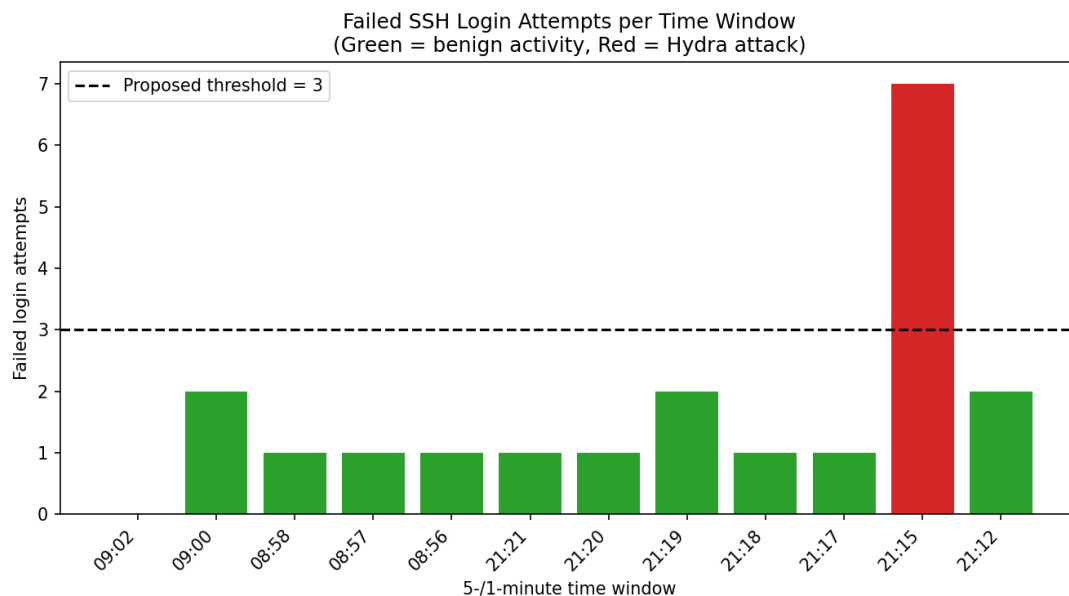


Figure 8: Failed login attempts per one-minute window across the full test period. Benign windows (green) never exceeded 2 failed attempts; the Hydra attack window (red) produced 7.

Additional benign trials, including attempts using a non-existent username to simulate a realistic typo scenario, were also captured and are shown below alongside the raw Splunk aggregation used to compile Table 1.

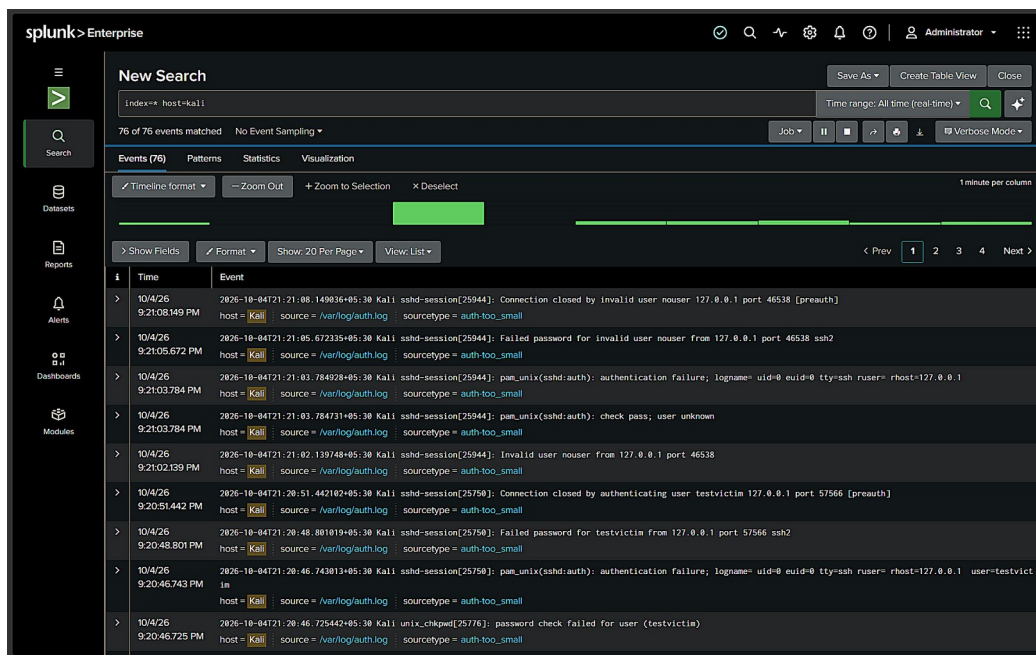


Figure 9: Raw Splunk event list showing a benign login attempt with an invalid username ('nouser'), captured as sshd-session connection-closed and invalid-user log entries, distinct in structure from the brute-force pattern.

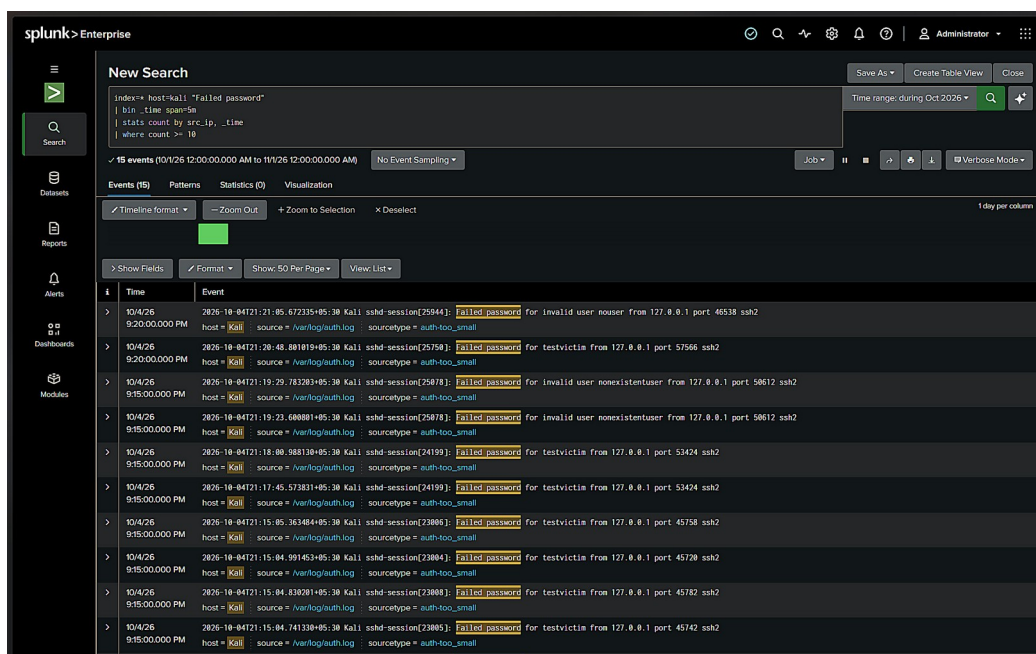


Figure 10: The core detection query (Section 6.7) applied to the dataset, returning 15 raw matching events later aggregated by time window to produce Table 1.

8. Threshold Tuning and Analysis

Using the captured dataset, the detection rule "count >= threshold" was evaluated at several threshold values to measure the trade-off between catching the real attack (true positives) and incorrectly flagging benign activity (false positives).

Threshold	False positives (of 11 benign windows)	Attack caught?
≥ 1	10 (91%)	Yes
≥ 2	3 (27%)	Yes

≥ 3	0 (0%)	Yes
≥ 5	0 (0%)	Yes
≥ 10	0 (0%)	No (missed — attack peaked at 7)

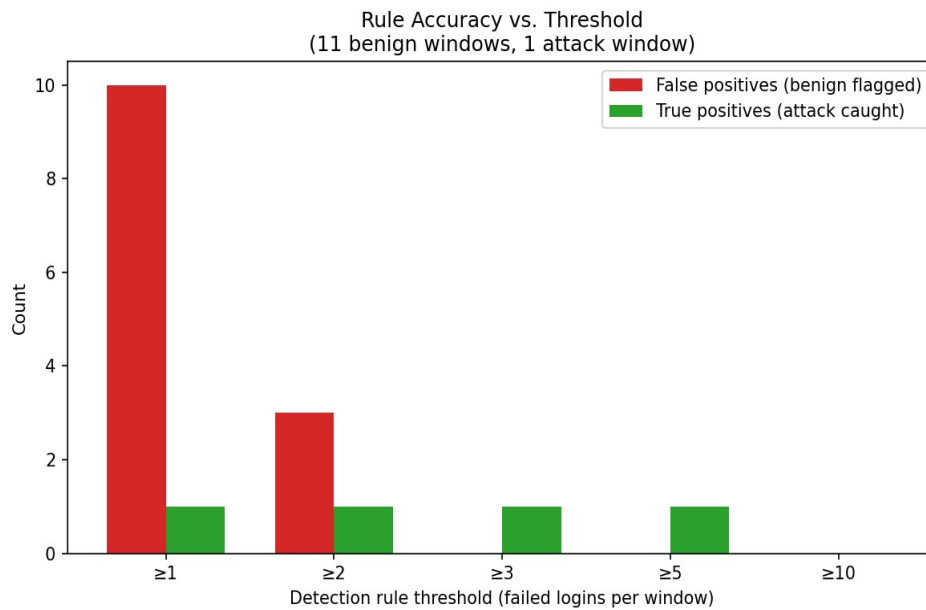


Figure 11: Detection accuracy at increasing thresholds. A threshold of 1 catches the attack but incorrectly flags 91% of benign windows. A threshold of 10 eliminates all false positives but misses the attack entirely, since this particular Hydra run peaked at 7 failed attempts. A threshold of 3 is the first value in this dataset that achieves zero false positives while still catching the attack.

This result directly illustrates the central problem in detection engineering: a rule tuned purely to "catch everything" (threshold ≥ 1) generates unacceptable noise, while a rule tuned purely to "avoid false alarms" (threshold ≥ 10) can miss a real attack if the attacker's behaviour falls below the chosen bar. The threshold of 3 represents the tuned, balanced rule for this specific dataset.

9. Discussion

The experiment confirms, on a small scale, the industry-reported problem of alert fatigue: an untuned, overly sensitive rule (flagging on a single failure) would have generated ten false alerts for every one genuine attack in this dataset — a 10:1 noise-to-signal ratio that would be unsustainable for a human analyst reviewing alerts manually at scale. Tuning the rule to a threshold of 3 removed all observed false positives without sacrificing detection of the attack, demonstrating that a modest, data-driven adjustment can meaningfully change an analyst's daily alert burden.

It is also notable that one benign trial (21:18) involved a failed attempt followed immediately by a successful login — structurally identical in shape to the attack pattern (failures then success), differing only in volume. This confirms that volume-based thresholds, not simply the presence of a failure-then-success pattern, are what separates benign retries from brute-force attacks in this dataset, reinforcing why naive Sigma-style rules that trigger on any failure-then-success sequence would be too aggressive without a minimum failure count.

10. Limitations

- All traffic originated from a single source IP (127.0.0.1), since attacker and defender shared one VM; a distributed or multi-source attack, or genuinely separate attacker/victim machines, was not tested
- The benign dataset (11 windows) and attack dataset (effectively 2 distinct Hydra runs) are small; a production environment would require a much larger sample, including real user traffic over days or weeks, to set a statistically robust threshold
- SSH's own MaxAuthTries setting disconnected sessions after a small number of failures, which constrained how many consecutive failures a single benign session could generate — a real attacker using a dedicated tool like Hydra is not subject to this same per-session limit, since it opens a fresh connection for each attempt
- The experiment used one specific attack tool (Hydra) and one specific technique (SSH brute force); results may not generalise to other protocols or slower, deliberately evasive attack patterns

11. Legal Framework: Information Technology Act, 2000

All attack simulation in this assignment was performed exclusively against a locally hosted virtual machine and a test account created specifically for this purpose, with no access to any system, network, or account not owned and controlled by the author. Unauthorised access to a computer system in India is addressed under Section 43 (civil liability for unauthorised access or damage) and Section 66 (criminal liability for the same conduct done dishonestly or fraudulently) of the Information Technology Act, 2000. Confining all testing to an isolated, personally owned lab environment ensured this assignment's practical component remained fully compliant with this framework.

12. Build Log: Troubleshooting the Lab Environment

Building this lab was not a single uninterrupted process, and several real obstacles were encountered and resolved along the way. These are documented here both for transparency and because the troubleshooting process itself demonstrates practical skills (network diagnostics, Linux log configuration, firewall administration) directly relevant to a SOC analyst role. Each issue below is presented as encountered, with the diagnostic step and the fix.

12.1 Issue: Missing /var/log/auth.log on Kali

On first attempting to configure the Universal Forwarder to monitor /var/log/auth.log, the file did not exist. Recent Kali Linux releases rely on systemd-journald for logging by default and do not populate the traditional syslog-style /var/log/auth.log file out of the box. This was diagnosed by listing the contents of /var/log and confirming the file's absence, then resolved by installing and enabling the rsyslog package, which restores traditional plaintext log file output alongside journald:

```
sudo apt update sudo apt install rsyslog sudo systemctl enable rsyslog
sudo systemctl start rsyslog
```

After this change, /var/log/auth.log was created and began populating correctly with authentication events, confirmed via `ls -la /var/log/auth.log`.

12.2 Issue: Host IP Address Drift (DHCP)

The lab's host machine and Kali VM were connected via a bridged VirtualBox network adapter, with both devices receiving IP addresses dynamically via DHCP from the home router. Across multiple work sessions on different days, the host machine's IP address changed (observed values included 192.168.1.110, 192.168.1.109, and 192.168.1.106 on different days), which silently broke the forwarder's configured destination, since the Universal Forwarder's forward-server setting is a static IP:port pair, not a hostname.

This was diagnosed by running `ipconfig` on the host machine and comparing the active Wi-Fi adapter's IPv4 address against the address the Kali forwarder was currently configured to send to, retrieved via:

```
sudo /opt/splunkforwarder/bin/splunk list forward-server -auth
admin:<password>
```

Each time a mismatch was found, the forwarder was repointed at the current host IP using `splunk add forward-server <new-ip>:9997`, followed by a forwarder restart. This is documented as a known limitation in Section 10, with a static IP or router-level DHCP reservation identified as the permanent fix.

12.3 Issue: Windows Firewall Blocking Inbound Port 9997

After correcting the IP address, the forwarder reported a successful connection, but no events appeared in Splunk's search interface. Diagnosis involved checking the host machine's active network profile in Windows Defender Firewall, which revealed the host's network was classified as a Public network (rather than Private), under which Windows blocks unsolicited inbound connections by default — including the forwarder's connection attempt on TCP port 9997.

This was resolved by creating an explicit inbound firewall rule permitting TCP traffic on port 9997 across all three Windows firewall profiles (Domain, Private, and Public), using Windows Defender Firewall with Advanced Security → Inbound Rules → New Rule → Port → TCP 9997 → Allow the connection. Once this rule was added, previously generated log data (including the synthetic test event shown in Figure 5, Section 6.4) appeared in Splunk within seconds.

12.4 Issue: SSH Session Disconnects Limiting Benign Trial Size

During the benign-traffic data collection phase (Section 6.2), an attempt to manually fail a login five or six times in a single SSH session before succeeding did not produce the expected number of failed-attempt log entries. Investigation of the resulting Splunk data showed that most manual trials produced only 1-2 failed attempts per session rather than the intended 4-6. This was determined to be caused by OpenSSH's default `MaxAuthTries` setting, which disconnects a client after a small number of failed authentication attempts within a single connection, forcing a reconnect to continue. Rather than treating this as a failed trial, it was retained as a genuine and relevant finding: it demonstrates that SSH itself provides a first layer of brute-force mitigation independent of any SIEM-level detection rule, a point incorporated into the Discussion (Section 9).

13. Conclusion

This assignment set out to test, rather than simply describe, the problem of alert fatigue in a SOC context. Using a small but genuine Splunk deployment and real attack and benign log data, the results show a measurable, visualisable trade-off between detection sensitivity and false-positive noise. A threshold-based brute-force detection rule that would have been 91% false positives at its most sensitive setting was reduced to zero false positives through evidence-based tuning, without losing the ability to detect the real attack — a concrete demonstration of the detection engineering process that underpins effective SOC alerting in practice.

14. Future Work

This lab's scope was intentionally limited to fit the assignment timeline, and several extensions would meaningfully strengthen the findings presented here if pursued further:

- Multi-source testing: repeating the attack simulation from a genuinely separate attacker VM against a separate victim VM (rather than both roles on one machine via loopback), to confirm the detection logic behaves identically with real network-layer traffic
- Distributed/low-and-slow attacks: simulating an attacker who deliberately spaces out login attempts (e.g., one attempt every 2-3 minutes) to test whether the tuned threshold of 3 per window can be evaded, and if so, what windowing strategy would be needed to catch it
- Larger benign dataset: collecting benign login data over several real days of ordinary use, rather than a single concentrated testing session, to build a more statistically robust baseline
- Cross-platform extension: adding a Windows Event Log data source (via Sysmon or the native Windows Event Log input) alongside the existing Linux data, to test whether the same threshold-tuning approach transfers to a different operating system's authentication telemetry
- Automated alerting: converting the manually-run SPL queries in this assignment into a saved Splunk alert with a scheduled search and a notification action, to demonstrate the full detection-to-alerting pipeline rather than ad-hoc querying
- Comparative SIEM testing: running the identical dataset and detection logic through a second SIEM platform (e.g., Wazuh) to compare default out-of-the-box detection behaviour against Splunk's, building on the tool-comparison approach considered during this assignment's planning phase

15. Glossary of Terms

Term	Definition
SIEM	Security Information and Event Management — a platform that centralises, normalises, and allows analysis of security-relevant log data from multiple sources.
SOC	Security Operations Centre — the team (and/or facility) responsible for monitoring, detecting, and responding to security incidents in an organisation.
False positive	An alert generated by a detection rule for activity that is, on investigation, not actually malicious.
True positive	An alert generated by a detection rule that correctly corresponds to genuine malicious activity.
Alert fatigue	The desensitisation of analysts to security alerts caused by a sustained high volume of false positives, increasing the risk that genuine threats are missed or under-investigated.
Brute force attack	An attack technique that attempts many candidate passwords (or keys) in sequence against an authentication system until a correct one is found.
SPL	Search Processing Language — the query language used within Splunk to search, filter, and aggregate indexed log data.
Forwarder	A lightweight Splunk component installed on a monitored host that reads local log files and transmits new entries to a central Splunk indexer.
MITRE ATT&CK	A publicly maintained knowledge base of real-world adversary tactics and techniques, used by security teams as a common reference framework for describing and mapping threats and detections.
Threshold tuning	The process of adjusting the numeric trigger condition of a detection rule (e.g.,

	number of failed logins per time window) to balance detection sensitivity against false-positive rate.
--	--

16. References

- MITRE ATT&CK. T1110 – Brute Force. attack.mitre.org
- Splunk Inc. Splunk Search Processing Language (SPL) documentation. docs.splunk.com
- Splunk Inc. Splunk Universal Forwarder documentation. docs.splunk.com
- Splunk Inc. Splunk Enterprise Free license and installation guide. splunk.com
- THC-Hydra project (van Hauser / THC, David Maciejak). github.com/vanhauser-thc/thc-hydra
- Government of India, Ministry of Electronics and Information Technology. Information Technology Act, 2000 (as amended by the IT (Amendment) Act, 2008).
- SANS Institute. Alert Fatigue and SOC Analyst Burnout (SANS whitepapers and surveys on SOC operations).
- Kali Linux Documentation. Kali Linux Revealed — networking and logging configuration (rsyslog, systemd-journald). kali.org
- OpenBSD Project. sshd(8) and sshd_config(5) manual pages — MaxAuthTries and related authentication controls.

Appendix A: Lab Evidence (Screenshots)

The following screenshots were captured directly from the author's own lab during the course of this assignment and document each major stage of the experiment described in Sections 5-7, as well as the build and troubleshooting process described in Section 12.

Figure A-1: Test Account and SSH Service Configuration

```
(zxb@Kali)-[~/Downloads]
$ id testvictim
uid=1002(testvictim) gid=1002(testvictim) groups=1002(testvictim),100(users)

(zxb@Kali)-[~/Downloads]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-10-04 20:55:54 IST; 3min 7s ago
  Invocation: a624bf3b53df44a4b5d3a1fe3793841f
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 13640 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 13642 (sshd)
       Tasks: 1 (limit: 12230)
      Memory: 1.8M (peak: 2.8M)
         CPU: 60ms
    CGroup: /system.slice/ssh.service
            └─13642 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Oct 04 20:55:54 Kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Oct 04 20:55:54 Kali sshd[13642]: Server listening on 0.0.0.0 port 22.
Oct 04 20:55:54 Kali sshd[13642]: Server listening on :: port 22.
Oct 04 20:55:54 Kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

The testvictim account created for this lab, and confirmation that the OpenSSH server (sshd) is enabled and actively listening on port 22 on the Kali VM.

Figure A-2: Successful Hydra Brute-Force Attack

```
File Machine View Input Devices Help
zxb@Kali: /var/log

Session Actions Edit View Help

(zxb@Kali)-[/var/log]
$ ls
alternatives.log  boot.log.3  gvm  macchanger.log.1.gz  private  user.log  Xorg.0.log
alternatives.log.1  btmap  inetmgr  macchanger.log.2.gz  redis  vboxadd-install.log  Xorg.0.log-old
apache2  btmap.1  installer  mosquitto  runit  vboxadd-setup.log  Xorg.0.log-old
apt  cron.log  journal  mysql  samba  vboxadd-setup.log.1  Xorg.0.log-old
auth.log  dpkg.log  kern.log  nginx  speech-dispatcher  vboxadd-setup.log.2  Xorg.0.log-old
boot.log  dpkg.log.1  lastlog  notus-scanner  stunnel  vboxadd-setup.log.3  Xorg.0.log-old
boot.log.1  dpkg.log.2.gz  lightdm  openvpn  syslog  vboxadd-setup.log.4  Xorg.0.log-old
boot.log.2  fontconfig.log  macchanger.log  postgresql  sysstat  wtmp

(zxb@Kali)-[/var/log]
$ sudo rm testing.txt
rm: cannot remove 'testing.txt': No such file or directory

(zxb@Kali)-[/var/log]
$ echo "$(date +%b %d %H:%M:%S) Kali sshd[9999]: Failed password for testvictim from 127.0.0.1 port 5000000 ssh2" | sudo tee -a /var/log/auth.log
Oct 10/04/26 21:12:10 Kali sshd[9999]: Failed password for testvictim from 127.0.0.1 port 5000000 ssh2

(zxb@Kali)-[/var/log]
$ echo "$(date +%b %d %H:%M:%S) Kali sshd[9999]: Failed password for testvictim from 127.0.0.1 port 5000000 ssh2" | sudo tee -a /var/log/auth.log
Oct 10/04/26 21:12:41 Kali sshd[9999]: Failed password for testvictim from 127.0.0.1 port 5000000 ssh2

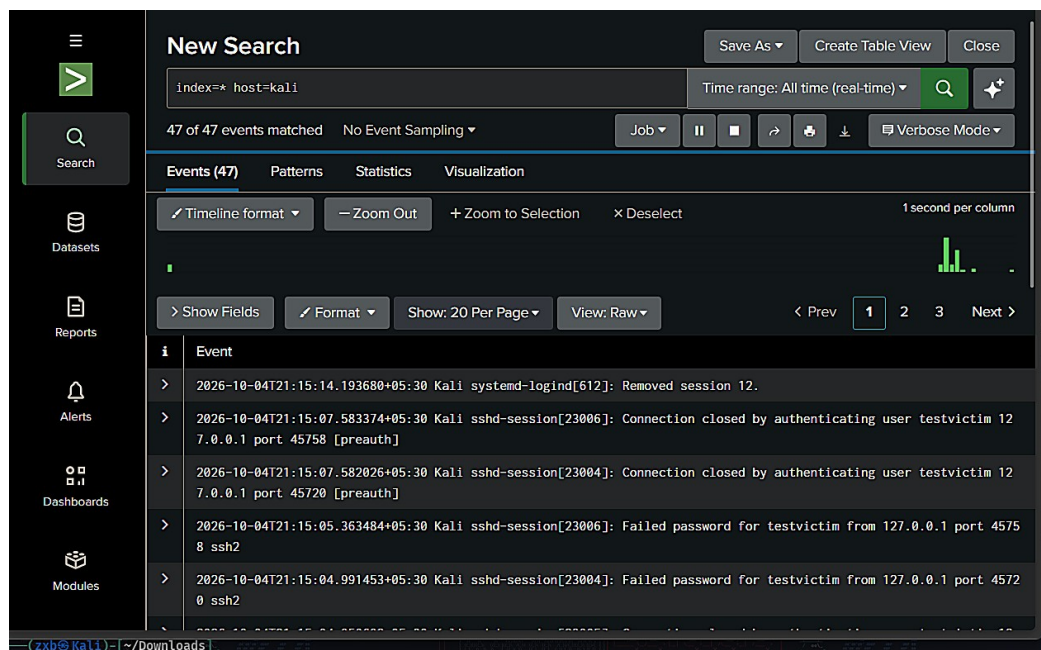
(zxb@Kali)-[/var/log]
$ hydra -l testvictim -P password.txt ssh://127.0.0.1
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-04 21:14:20
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[ERROR] File for passwords not found: password.txt

(zxb@Kali)-[/var/log]
```

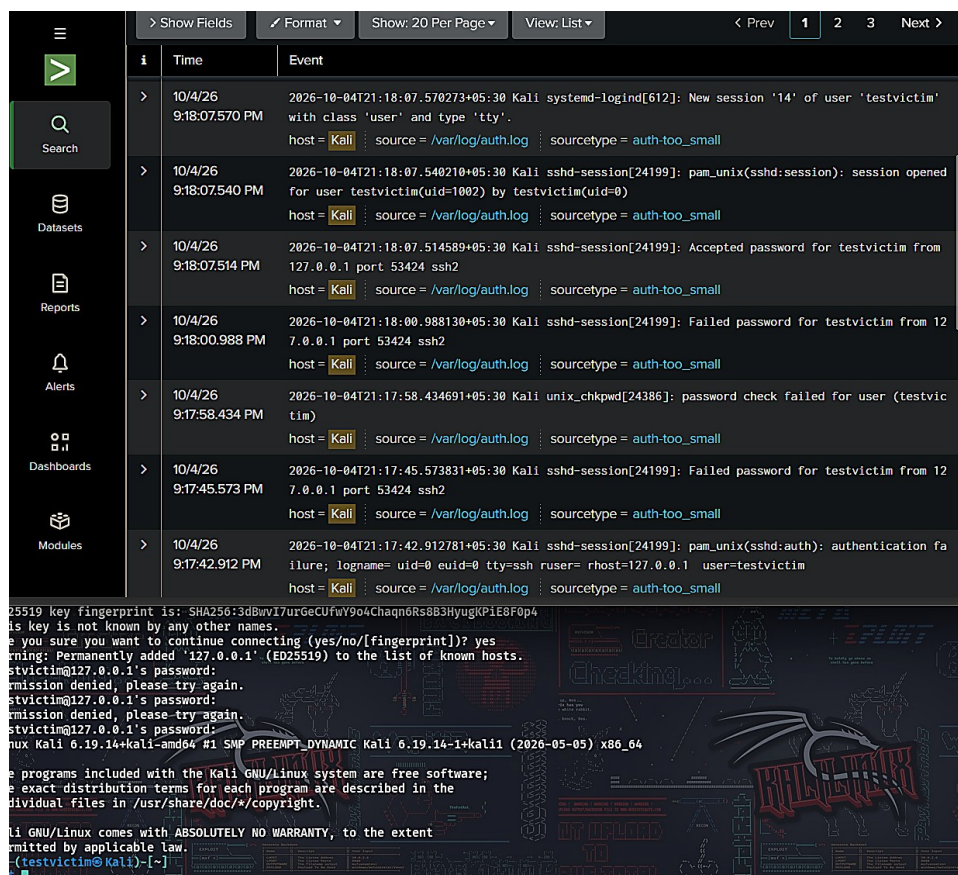
Hydra completing an online brute-force attack against the testvictim SSH account, correctly recovering the password 'password123' from the supplied wordlist (1 of 1 target successfully completed).

Figure A-3: Attack Traffic Captured in Splunk



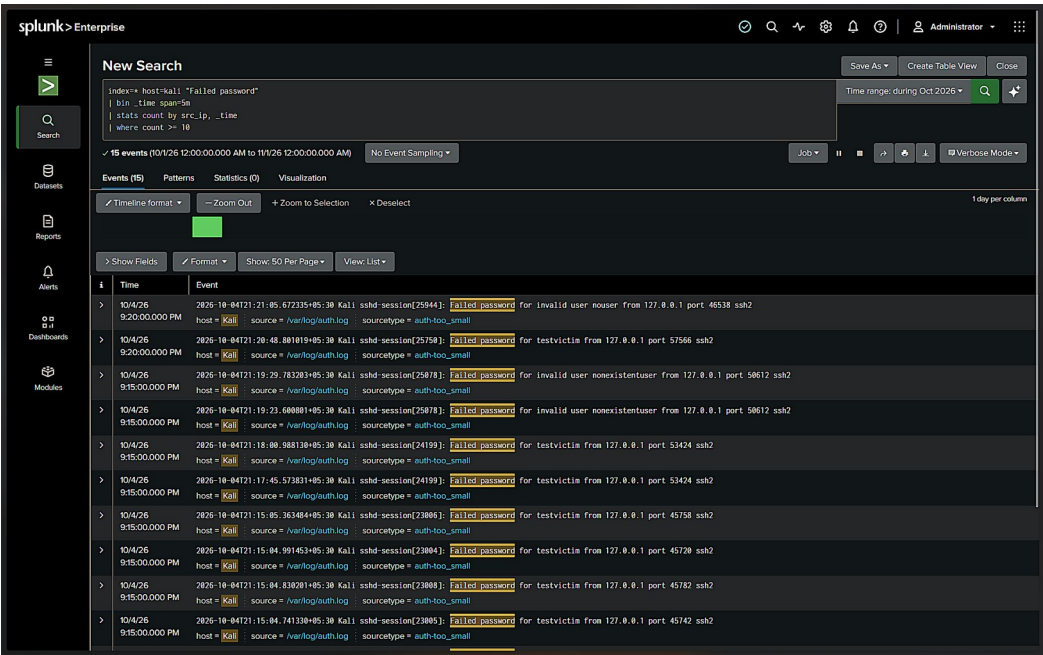
Splunk's Search & Reporting interface showing 47 ingested events from the Kali forwarder, including the real sshd session logs generated by the Hydra attack, confirming end-to-end log delivery from VM to SIEM.

Figure A-4: Benign Login Activity Captured in Splunk



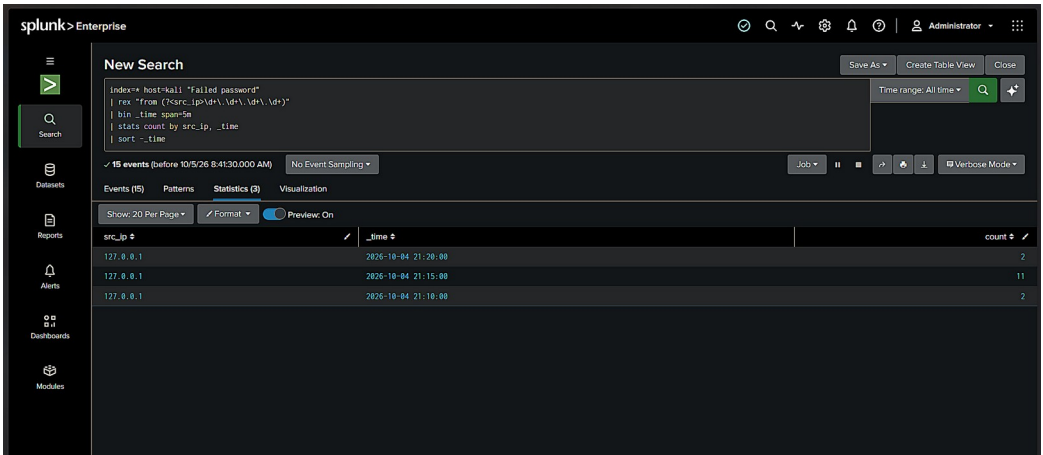
A broader Splunk search (76 events) showing both attack traffic and manually generated benign login attempts, including an invalid-username attempt ('Invalid user nouser') used to simulate a realistic typo scenario.

Figure A-5: Detection Query Applied to Captured Data



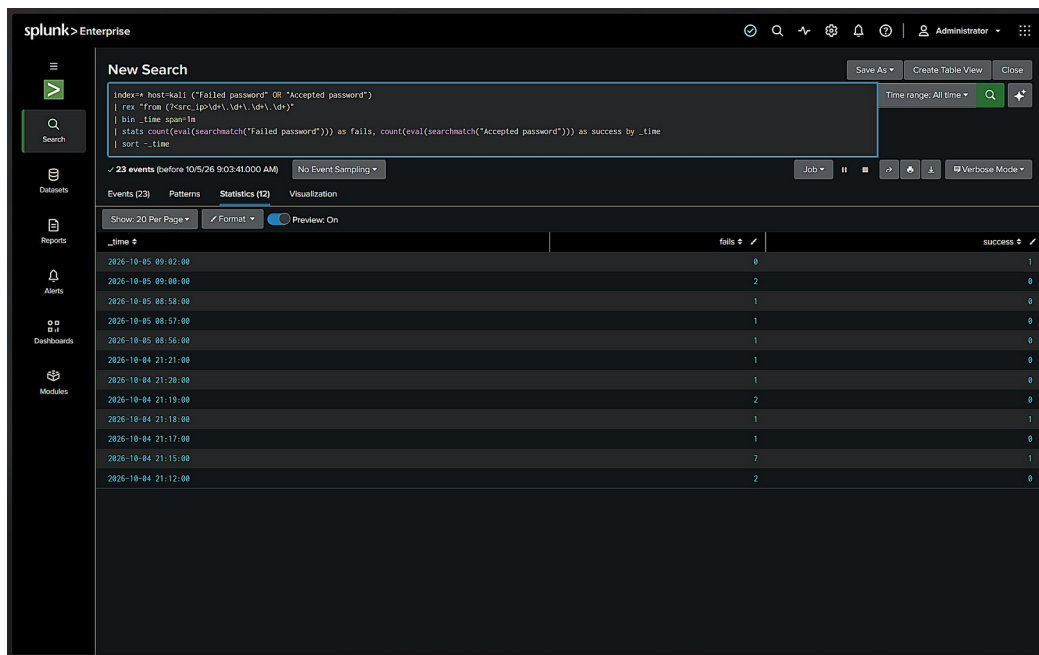
The SPL detection query (Section 6.5) applied against the dataset, isolating 'Failed password' events for further aggregation by source IP and time window.

Figure A-6: Aggregated Statistics — Failed Logins per Time Window



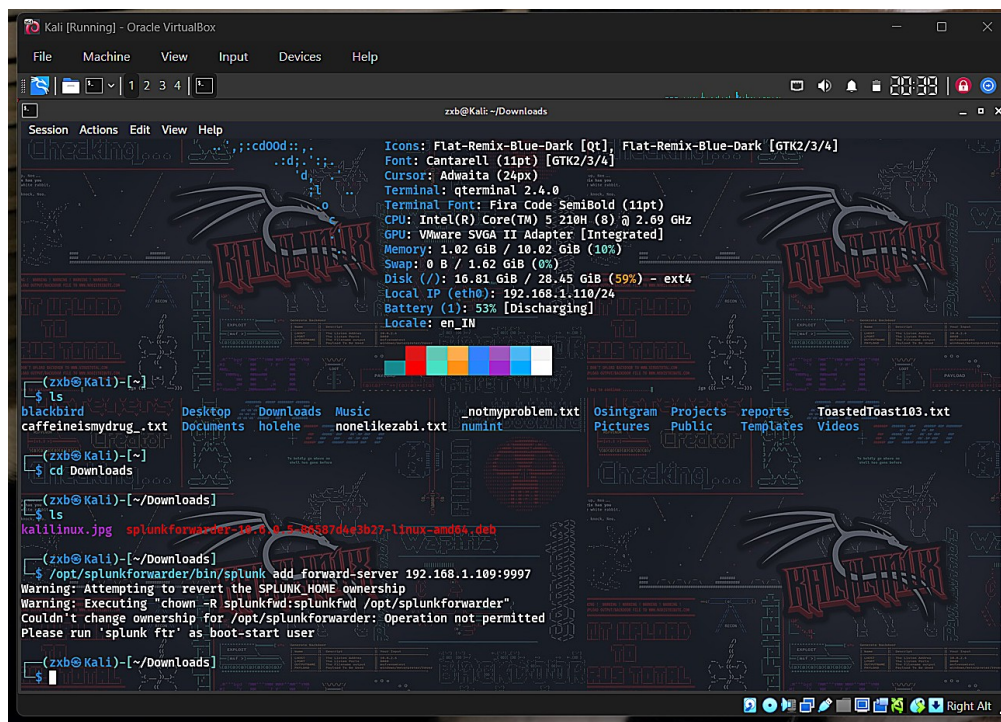
Splunk's Statistics tab showing the stats count by src_ip, _time aggregation, revealing the three key data points (2, 11, and 2 failed attempts) that first demonstrated a measurable gap between benign activity and the Hydra attack burst.

Figure A-7: Complete Dataset with Success/Failure Correlation



The final aggregated dataset (Section 7, Table 1) combining failed and successful login counts per time window across all benign and attack trials, used to generate Figures 8 and 11 in the main body of this report.

Figure A-8: Kali VM System Configuration



System summary of the Kali Linux VM used throughout this lab (CPU, memory, disk, and local IP), captured via the terminal's fastfetch-style system banner, documenting the exact environment the experiment was run in.

Figure A-9: Forwarder Reconnection After IP Drift

A terminal window on a Kali Linux system with a dark, stylized background. The user is in the directory ~/Downloads. They run a command to add a forward server. The terminal shows the command, a password prompt, a warning about ownership, a login prompt, and the successful addition of the forward server. Then, they run a command to add a monitor input. The terminal shows the command, the same warning, and an error message stating that the input already exists.

```
(zxb@Kali)-[~/Downloads]
$ sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168.1.109:9997 -auth
[sudo] password for zxb:
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
Your session is invalid. Please login.
splunk username: ToastedToast
Password:
Added forwarding to: 192.168.1.109:9997.

(zxb@Kali)-[~/Downloads]
$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/auth.log -auth
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
cannot create another input with the name "/var/log/auth.log", one already exists.

(zxb@Kali)-[~/Downloads]
$
```

The Universal Forwarder on Kali being reconfigured with a corrected host IP address after a DHCP-driven IP change was diagnosed, as described in the build log (Section 12.2) — one of several such reconnections required across different work sessions.

Figure A-10: Full Dataset After Firewall Fix (192 Events)

Not reproduced here a second time — see Figure 4 (Section 5.4) for the equivalent full-pipeline confirmation screenshot, captured once the Windows Firewall inbound rule for port 9997 (Section 12.3) had been applied.